Estruturas de Programação

04

Vamos começar a programar, utilizando as técnicas mais comuns como: estrutura de controle, repetição, funções e etc.

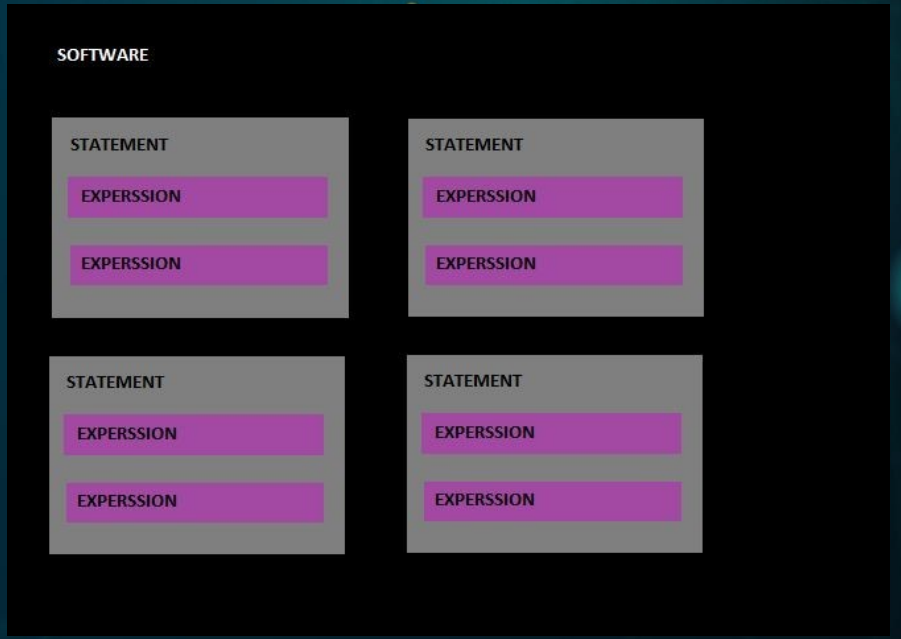
O que é um programa/software?

- É um conjunto de declarações (statements);
- Statements são conjuntos de expressões (expressions);
- Expression é todo fragmento de código que produz um valor;



O que é um programa/software?

- Software tem um ‘objetivo’;
- Statements ‘guiam’ o software para seu objetivo;
- Expressions são os valores que os statements esperam para guiar o software;



O que é um programa/software?

- Nós já vimos statements e expressions de forma simples;
- Statement: console.log(1 > 2);
- Expression: console.log('batata');
- Porém ainda não é o suficiente para criar um software;
- E é essa junção dos conceitos nosso objetivo nesta seção! =)



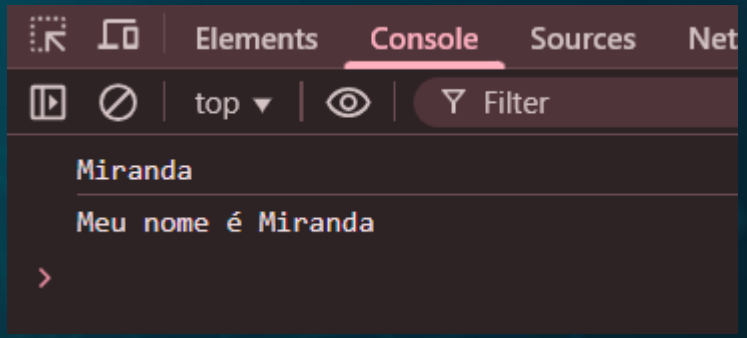
Como salvar valores na memória

- Salvamos os valores em variáveis (isso é um statement);
- let laranjas = 5;
- let é a nova forma de declarar variáveis a partir da versão ES6;
- E depois podemos criar expressões com os valores salvos;
- console.log(laranjas * laranjas);
- console.log(`Tem ${laranjas} na sexta`);



Como salvar valores na memória

- Exercício:
- Escreva seu nome;
- Use literals com a frase Meu nome é





Como salvar valores na memória

- Podemos também mudar o valor que foi salvo anteriormente;
- laranjas = laranjas + 1;
- console.log(laranjas);
- Um let pode definir várias variáveis também;
- let quadrado = 4, triangulo = 3, retangulo = 4;
- console.log(quadrado * (triangulo + retangulo));



Como salvar valores na memória

1_variaveis > JS scripts.js > ...

```
1   let nome = "André";
2   console.log(nome);
3   console.log(`O meu nome é ${nome}`);
4
5   let laranjas = 5;
6   console.log(laranjas * laranjas);
7
8   nome = "João";
9   console.log(nome);
10
11  laranja = 83284;
12  console.log(laranja);
13
14  laranja = "Laranja";
15  console.log(laranja);
```

Filtrar saída

```
André
O meu nome é André
25
João
83284
Laranja
```



Outras maneiras de salvar valor

- Podemos definir valores com var e const;
- var nome = ‘Pedro’;
- console.log(nome);
- const sobrenome = ‘Soares’;
- console.log(sobrenome);
- Testar:

  var nome = “Miranda”;

  const ip = “127.0.0.1”;
- Logo após testar tente alterar o valor do ip e

```
Miranda
127.0.0.1
▶ Uncaught TypeError: invalid assignment to const 'ip'
    <anonymous>  …o/Redes de Computadores/DEV WEB II/A
  [Saiba mais]
```



Outras maneiras de salvar valor

- var é uma forma mais antiga, você deve optar por let ou const;
- const vem de constante, ou seja, o seu valor não pode ser alterado;
- Para fazer com que uma variável receba um dado, utiliza-se o conceito de atribuição. Em JavaScript, a atribuição de valor de uma variável é feita com o sinal "=".
- Exemplo: const idade = 18
- Mais a frente no curso, veremos em detalhes diferenças de var, let e const;



Variáveis e Constantes

As variáveis declaradas em um programa devem possuir um nome, seguindo algumas regras de nomenclatura. Em JavaScript, os nomes de variáveis não podem:

- Conter espaços;
- Começar por número;
- Conter caracteres especiais, como: +, -, *, /, %, (,), {, }, !, @, #;
- Utilizar nomes de palavras reservadas da linguagem, como *function*, *var*, *new*, *for* ou *retorn*.

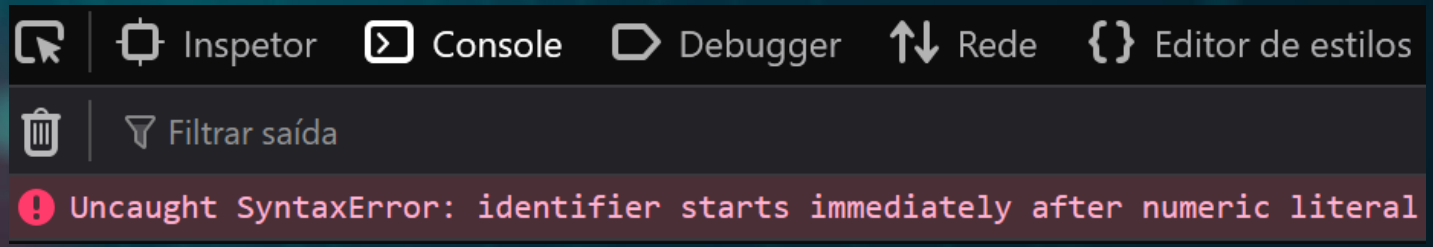


Obs: Variáveis escritas com letras maiúsculas são diferentes de variáveis escritas com letras minúsculas.

Convenção no nome das variáveis

- Não pode começar com um número (let 2teste);
- Mas pode terminar com número (let teste9 = ‘testando’)
- Pode ter $ ou _, mas não outros caracteres especiais (let $nome, _nome);
- Mas não pode ter pontuação ou outros especiais (let @teste)
- Podemos iniciar com letra maiúscula (let Nome = ‘Matheus’);
- Ou usar camelCase (let meuPrimeiroNome = ‘Matheus’);

  (Modelo mais utilizado pelos programadores. Mais indicado!)
- Testar: let 3nome = ‘Teste’;





Convenção no nome das variáveis

- Verdadeiro ou Falso:

let 3nome = 'Teste';

let nome3 = 'Teste1';

let $nome = 'Teste2';

let _nome = 'Teste3';

let !nome = 'Teste4';

let Nome = 'Teste5';

let @nome = 'Teste6';

let seuNome = 'Teste7';

3_convencao > JS script.js > ...

```
1   let 3nome = 'teste';
2   let nome3 = 'teste1';
3   let $nome = 'teste2';
4   let _nome = 'teste3';
5   let !nome = 'teste4';
6   let Nome = 'teste5';
7   let @nome = 'teste6';
8   let seuNome = 'Teste7';// mais indicado!
9   let seu_nome = 'teste8';
```



Nomes de variáveis reservados

- Alguns nomes não podem ser utilizados para iniciar variáveis, veja:
- break case catch class const continue debugger default delete do else enum export extends false finally for function if implements import interface in instanceof let new package private protected public return static super switch this throw true try typeof var void while with yield
- Ex: let break = ‘parar’;
- Porém podemos usar elas + alguma palavra/digito:
- Ex: let breakMatheus = 1;

Nomes de variáveis reservados

- Testar as variáveis abaixo:

  let if = ‘Teste’;

  let function = ‘Teste’;

  let functionTest = ‘Teste 1’;

  let function1 = ‘Teste 2’;

  console.log (functionTest);

  console.log (function1);

4_reservadas > JS script.js > ...

```
1   let if = 'teste1';
2   let function = 'teste2';
3   let functionTest = 'teste3';
4   let function1 = 'teste4';
5
6   console.log(functionTest);
7   console.log(function1);
```

Como funciona o ambiente

- Quando qualquer programa é iniciado, um ambiente é criado;
- E este ambiente não inicia vazio
- Ele contém **funções** nativas da linguagem;
- **Funções** são blocos de código, que nos retornam um valor ou ação;
- O ambiente de JavaScript, neste curso, poderiamos entender como o navegador



A estrutura de uma função

- Antes de mais nada: funções são chamadas durante o programa;
- Você também pode ouvir: executar, chamar, invocar uma função;
- E nós chamamos a função utilizando o nome dela + abrindo e fechando parênteses;
- .

```
nomeDaFuncao();
```



A estrutura de uma função

- Também podemos inserir parâmetros, em algumas funções são obrigatórios;
- Basicamente são valores que podem mudar o resultado da função;

```
nomeDaFuncao(parametro1, parametro2);
```

let idade = prompt (“Qual a sua discilina favorira?”);



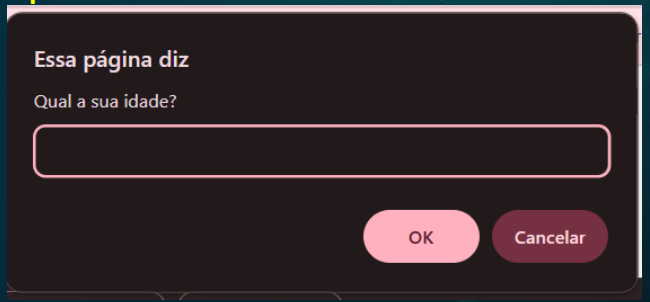
Funções built-in

Funções built-in (internas) em JavaScript são métodos e objetos globais nativos da linguagem, disponíveis para uso imediato sem necessidade de importação. Elas facilitam tarefas comuns como manipulação de strings , números, arrays e operações matemáticas (Math.random), garantindo desempenho e padronização.



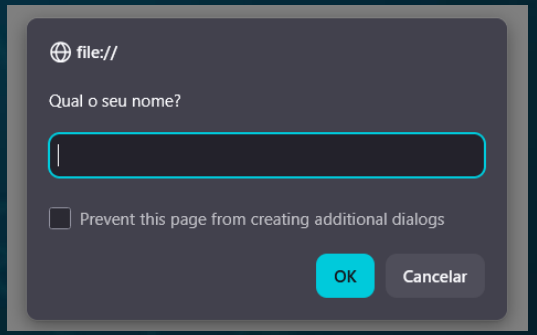
Função built-in: prompt()

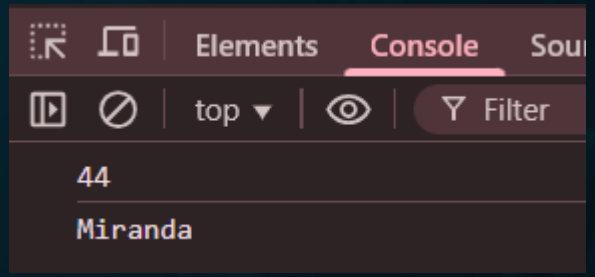
- Recebe um input do usuário do sistema e pode guardar este valor;
- Ex: let idade = prompt('Qual sua idade?');
- console.log(idade);
- Esta função é pouco utilizada.(Invasiva, não ter a possibilidade de alterar e má pratica)
- Teste: Crie prompts para ler a idade e o nome...



5_prompt > JS script.js > ...

```
1  let idade = prompt ("Qual a sua idade?");
2  console.log(idade);
3
4  let nome = prompt ("Qual o seu nome?");
5  console.log(`O seu nome é ${nome}`);
```



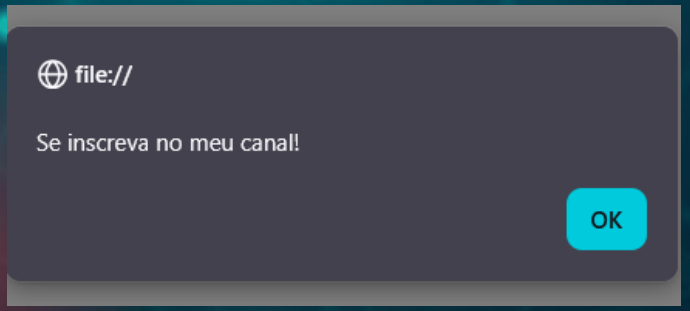


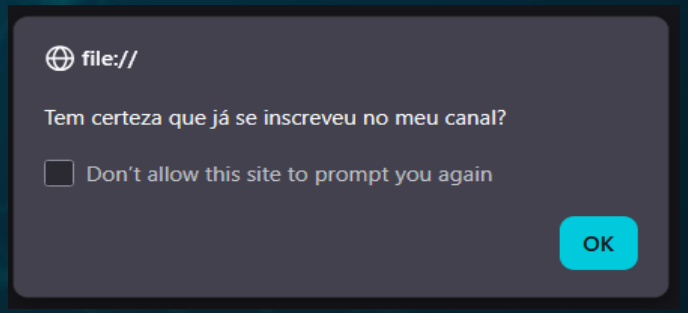
Função built-in: alert()

- Exibe uma mensagem na tela do usuário;
- Ex: alert(‘Veja esta mensagem!’);
- Esta função é pouco utilizada. (mesma situação da função prompt)
- Testar:

alert ("Se inscreva no meu canal");

alert ("Tem certeza que já se inscreveu no meu canal?");







Função built-in: Math.x()

- Função utilizada para expressões/cálculos matemáticos;
- Biblioteca nativa de matemática do JS;
- Ex: let maiorNumero = Math.max(1,5,2,3);
- console.log(maiorNumero);
- Esta função é muito utilizada.
- Testar para maior,menor;
- Arredondar e arredondarParaCima o número 3.47989898;

```
1   let maior = Math.max (6,12,28,55,8);
2   console.log(maior);
3
4   let menor = Math.min (6,12,28,55,8);
5   console.log(menor);
6
7   let arredondar = Math.round (5.2934723);
8   console.log(arredondar);
9
10  let arredondarParaCima = Math.ceil (5.2934723);
11  console.log(arredondarParaCima);
```



Função built-in: Math.x()

Principais Recursos do Objeto `Math` (Nativo)

- **Arredondamento:**
  - `Math.round(x)` : Arredonda para o inteiro mais próximo.
  - `Math.ceil(x)` : Arredonda para cima.
  - `Math.floor(x)` : Arredonda para baixo.
- **Aritmética e Aleatoriedade:**
  - `Math.random()` : Retorna um número pseudoaleatório entre 0 e 1.
  - `Math.pow(base, exp)` : Potência ($base^{exp}$).
  - `Math.sqrt(x)` : Raiz quadrada.
  - `Math.abs(x)` : Valor absoluto.
- **Máximos e Mínimos:**
  - `Math.max(a, b, ...)` : Retorna o maior valor.
  - `Math.min(a, b, ...)` : Retorna o menor valor.
- **Trigonometria:** `Math.sin()` , `Math.cos()` , `Math.tan()` , etc. MDN Web Docs +4



Função built-in: console.log()

- Exibe uma mensagem na tela, que é o argumento que passamos;
- Ex: console.log('exibindo esta mensagem!');
- Esta função é muito utilizada.
- Veremos como fazer as nossas funções mais a frente;
- Testar: Escreva sua idade, em seguida seu nome e usando o literals use a frase Meu nome é …., e tenho … anos.

```
1   let idade = 45;
2   let nome = "Miranda";
3
4   console.log(idade);
5   console.log (nome);
6   console.log(`Meu nome é ${nome} e tenho ${idade} anos`);
```

```
45
Miranda
Meu nome é Miranda e tenho 45 anos
```

